# Halving the Constant in the Frank–Wolfe Rate via a Variance Argument

Samson Fraser

October 22, 2026

**Abstract**

The Frank–Wolfe method minimizes a smooth convex function over a convex set by repeatedly minimizing a linear function over the set and stepping toward the answer. It never projects onto the set, which makes it attractive when projecting is expensive, as on the nuclear-norm ball. With the standard step size $2/(k+2)$, its textbook guarantee says that after $N$ steps the error is at most $2LD^2/(N+2)$, where $L$ measures how fast the gradient changes and $D$ is the diameter of the set. No method of this kind can beat $LD^2/(4N)$, so only the constant is in question. Numerical worst-case computations by Taylor, Hendrickx and Glineur (2017) show that the textbook constant is two to three times too large, but no proof of a smaller constant has been published. We prove

$$f(x_N) - f^\star \;\le\; LD^2\,\frac{N+\frac{1}{3}H_{N-1}}{N(N+1)},$$

where $H_m$ is the $m$-th harmonic number. The bound is exact for $N=1$ and tends to $LD^2/N$, so the number of iterations needed to guarantee a given accuracy falls by about half: for an error of 1% of $LD^2$, from 198 to 101. The proof rests on one observation. Each iterate is an average of earlier points, and the more spread out those points are, the shorter the next step must be. Long steps now force short steps later. We also lower the constant for the Frank–Wolfe gap from 27/4 to 3, and report computations indicating that the exact worst-case constant is about 0.9.

## 1 Introduction

### 1.1 The method

Many optimization problems ask for the best point in a set that is easy to describe but awkward to move around in. Suppose we want to fill in the missing entries of a large table, such as users' ratings of films, with a matrix of small nuclear norm. A projected gradient method must project onto the nuclear-norm ball at every step, which costs a full singular value decomposition. Minimizing a *linear* function over the same ball needs only the top pair of singular vectors. The Frank–Wolfe method [1], also called the conditional gradient method, is built for this situation: it touches the set only through linear minimization.

Let $C \subset \mathbb{R}^d$ be compact and convex with Euclidean diameter $D$, and let $f$ be convex and $L$-smooth on $C$, that is, $\|\nabla f(x) - \nabla f(y)\| \le L\|x-y\|$ for $x, y \in C$. Let $f^\star = \min_C f$, attained at $x^\star$. The Frank–Wolfe method with open-loop step size starts from $x_0 \in C$ and iterates

$$s_k \in \operatorname*{argmin}_{s\in C} \langle \nabla f(x_k), s\rangle, \qquad x_{k+1} = x_k + h_k(s_k - x_k), \qquad h_k = \frac{2}{k+2}. \tag{1}$$

In words: at $x_k$ the method replaces $f$ by its tangent plane, finds the point $s_k$ of $C$ where that plane is lowest, and moves a fraction $h_k$ of the way toward it. On the probability simplex, $s_k$ is the vertex $e_i$ for the coordinate $i$ with the smallest partial derivative. On the nuclear-norm ball of radius $\tau$, it is $-\tau uv^\top$, where $(u, v)$ is the top singular pair of the gradient.

Unrolling (1) shows what the iterates look like. Since $h_0 = 1$, the starting point is forgotten after one step, and for $k \geq 1$

$$x_k = \sum_{j=0}^{k-1} \frac{2(j+1)}{k(k+1)}\, s_j. \tag{2}$$

Each iterate is a weighted average of the points the oracle has returned, with later points weighted more. After $k$ steps on the simplex, $x_k$ has at most $k$ nonzero entries; on the nuclear-norm ball, rank at most $k$. These sparse or low-rank iterates are often a reason to use the method in their own right; see [4] for applications.

### 1.2 What the rate says

The classical guarantee for (1) is [2]

$$f(x_N) - f^\star \leq \frac{2LD^2}{N+2}.$$

Read the right-hand side in two parts. The quantity $LD^2$ is the natural unit of error: $L$ says how sharply $f$ can curve and $D$ how far apart two points of $C$ can be, and a quadratic of curvature $L$ can change by $LD^2/2$ across $C$. The rest, $2/(N+2)$, is a pure number. It does not depend on the dimension, on $f$ or on $C$. It is a worst-case promise that holds for every problem in the class, which is what lets us budget iterations before running the method. With the classical bound, guaranteeing an error of at most 1% of $LD^2$ takes 198 iterations.

The order $1/N$ cannot be improved. Lan [7] showed that every method that accesses $C$ only through linear minimization needs error $LD^2/(4N)$ in the worst case. So the question left open is the constant: the best guarantee has the form $c\,LD^2/N$ for large $N$, and before this paper all we knew was $1/4 \leq c \leq 2$. Minor variants of the classical proof give $2LD^2/(N+3)$ [3, 4], and dual-averaging analyses give leading constants 2 or larger [5, 6].

### 1.3 Why the constant 2 has stood

The classical proof takes three lines. Smoothness and convexity give, at every step,

$$f(x_{k+1}) - f^\star \leq (1 - h_k)(f(x_k) - f^\star) + \tfrac{L}{2} h_k^2 \,\|s_k - x_k\|^2,$$

and bounding $\|s_k - x_k\|$ by $D$ and unrolling gives $2LD^2/(N+2)$. Each ingredient is sharp on its own. In particular a single step can have length exactly $D$. What the proof misses is that the steps cannot all be that long, and seeing why requires an argument about the whole history of the method rather than about one step.

Taylor, Hendrickx and Glineur [8] found the slack numerically. They cast the exact worst case after $N$ steps as a semidefinite program, a *performance estimation problem* (PEP), solved it for $N \leq 100$, and found values two to three times below the classical bound. Their abstract describes this as improving the standard guarantee by more than a factor of two; for Frank–Wolfe the improvement consists of these numerical values, with no closed-form bound. A numerical solution for each $N$ does not give a formula valid for all $N$. To our knowledge no proof of a

leading constant below 2 has been published, and the survey [4] and the recent PEP study [9] still list $2LD^2/(N+2)$ as the best known upper bound.

### 1.4 The idea

Picture the points $s_0, \dots, s_{k-1}$ as a cloud of masses, with mass $2(j+1)/(k(k+1))$ at $s_j$. By (2), the iterate $x_k$ is the cloud's center of mass. Let $V_k$ be the cloud's spread: the mass-weighted mean squared distance from its points to $x_k$. For any point $s$, the mean squared distance from $s$ to the cloud equals $\|s - x_k\|^2 + V_k$; this is the parallel-axis theorem of mechanics. Every point of the cloud lies in $C$, so it is within $D$ of $s_k$. Hence

$$\|s_k - x_k\|^2 \le D^2 - V_k.$$

A spread-out history forces a short step. In the other direction, a long step puts mass far from the center and spreads the cloud out. So the method cannot take full-length steps forever: it pays for each long step with shorter ones later. Figure 1 shows the picture. Keeping exact accounts of this trade-off gives our main result.

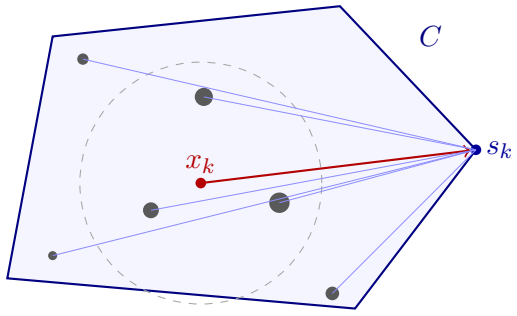



Figure 1: The iterate $x_k$ is the center of mass of the earlier oracle points (gray disks, sized by weight). Every gray point is within $D$ of $s_k$ (thin blue lines), and the mean of their squared distances to $s_k$ equals $\|s_k - x_k\|^2 + V_k$. So the step $s_k - x_k$ (red) can have length close to $D$ only if the cloud is tightly bunched, and each long step spreads the cloud.

### 1.5 Results

**Theorem 1.** *For every $N \ge 1$, the iterates (1) satisfy*

$$f(x_N) - f^\star \;\le\; LD^2\,\frac{N + \frac{1}{3}H_{N-1}}{N(N+1)}, \qquad H_m = \sum_{j=1}^{m} \frac{1}{j}, \quad H_0 = 0.$$

**Corollary 2.** *For every $N \ge 1$,*

$$f(x_N) - f^\star \le \frac{LD^2}{N+1}\Big(1 + \frac{H_{N-1}}{3N}\Big) \le \frac{4}{3}\,\frac{LD^2}{N+1}, \qquad \limsup_{N\to\infty} \frac{N\,(f(x_N) - f^\star)}{LD^2} \le 1.$$

In practical terms, the number of iterations that guarantees an error of at most $\varepsilon LD^2$ drops from about $2/\varepsilon$ to about $1/\varepsilon$; for $\varepsilon = 1\%$, from 198 to 101. Nothing about the algorithm changes. What changes is what we can promise about it.

The bound cannot be improved at $N = 1$. Take $f(x) = \frac{L}{2}x^2$ on $C = [0, D]$ and $x_0 = 0 = x^\star$. Then $\nabla f(x_0) = 0$, the oracle may return $s_0 = D$, and $f(x_1) - f^\star = LD^2/2$. For larger $N$ the bound is not exact. Section 4 compares it with the true worst case, computed by PEP: the true constant appears to be about 0.9, so Theorem 1 removes most of the slack in the classical bound but not all of it (Figure 2).

In practice one rarely knows $f^\star$, so solvers stop when the Frank–Wolfe gap, a computable upper bound on the error, is small. Section 3 improves the classical guarantee for this gap from $\frac{27}{4}LD^2/(N+2)$ to $3LD^2/(N+1)$. Section 2 proves Theorem 1.

## 2 Proof of Theorem 1

The proof has five steps. Step 1 is the classical one-step inequality. Step 2 tracks the spread of the cloud and turns the picture of Figure 1 into two exact relations. Step 3 handles the first step. Steps 4 and 5 combine the relations through a potential function and solve the resulting recursion in closed form.

By the scaling $x \mapsto x/D$, $f \mapsto f/(LD^2)$ we assume $L = D = 1$. Write $e_k = f(x_k) - f^\star$ for the error and $\delta_k = \|s_k - x_k\|^2$ for the squared step length.

**Step 1: descent.** By smoothness on the segment $[x_k, x_{k+1}] \subset C$, by the definition of $s_k$, and by convexity,

$$f(x_{k+1}) \le f(x_k) + h_k\langle \nabla f(x_k), s_k - x_k\rangle + \tfrac{h_k^2}{2}\delta_k, \qquad \langle \nabla f(x_k), s_k - x_k\rangle \le \langle \nabla f(x_k), x^\star - x_k\rangle \le -e_k,$$

so that

$$e_{k+1} \le (1 - h_k)e_k + \tfrac{h_k^2}{2}\,\delta_k. \tag{3}$$

The classical proof stops here and uses $\delta_k \le 1$.

**Step 2: the spread of the cloud.** Define probability measures $\mu_0 = \varepsilon_{x_0}$ and $\mu_{k+1} = (1 - h_k)\mu_k + h_k\,\varepsilon_{s_k}$, where $\varepsilon_y$ is the Dirac mass at $y$. By induction, $\mu_k$ is supported on a finite subset of $C$ and has mean $x_k$; for $k \ge 1$ it is the cloud of (2). Let $V_k = \mathbb{E}_{Y\sim\mu_k}\|Y - x_k\|^2$. For any point $s$, $\mathbb{E}\|s - Y\|^2 = \|s - x_k\|^2 + V_k$; taking $s = s_k \in C$ and using $\operatorname{diam} C = 1$,

$$\delta_k = \mathbb{E}_{Y\sim\mu_k}\|s_k - Y\|^2 - V_k \le 1 - V_k. \tag{4}$$

This is the first relation: spread limits step length. For the second, note that $x_{k+1} - x_k = h_k(s_k - x_k)$ and $s_k - x_{k+1} = (1 - h_k)(s_k - x_k)$. The law of total variance for the mixture $\mu_{k+1}$ gives

$$V_{k+1} = (1 - h_k)(V_k + h_k^2\delta_k) + h_k(1 - h_k)^2\delta_k = (1 - h_k)V_k + h_k(1 - h_k)\,\delta_k. \tag{5}$$

This is the second relation: step length builds spread. It is convenient to work with the room left, $W_k := 1 - V_k$, in which (4)–(5) become

$$0 \le \delta_k \le W_k, \qquad W_{k+1} = h_k + (1 - h_k)W_k - h_k(1 - h_k)\,\delta_k. \tag{6}$$

A long step (large $\delta_k$) uses up room, and room is what later steps need.

**Step 3: the first step.** Since $h_0 = 1$, we have $x_1 = s_0$ and $\mu_1 = \varepsilon_{s_0}$, hence $W_1 = 1$, and (3) with (4) ($V_0 = 0$) gives $e_1 \le \frac{1}{2}$. This is the case $N = 1$.

**Step 4: a potential.** Let $E_k = k(k+1)e_k$. Since $(k+1)(k+2)(1-h_k) = k(k+1)$ and $(k+1)(k+2)h_k^2/2 = 2(k+1)/(k+2)$, inequality (3) gives, for $k \geq 1$,

$$E_{k+1} \leq E_k + \frac{2(k+1)}{k+2}\,\delta_k. \tag{7}$$

A long step raises $E$ but uses up room $W$. We look for prices $\alpha_k \geq 0$ that make the potential $\Phi_k = E_k + \alpha_k W_k$ indifferent to the step length, so that the two effects cancel. Fix $N \geq 2$, set $\alpha_N = 0$, and for $k = N-1, \ldots, 1$ define

$$\alpha_k = \frac{2(k+1)}{k+2} + \alpha_{k+1}\Big(\frac{k}{k+2}\Big)^2. \tag{8}$$

**Lemma 3.** *$0 \leq \alpha_k \leq k+1$ for $2 \leq k \leq N$.*

*Proof.* Nonnegativity is clear. Use backward induction from $\alpha_N = 0$: if $\alpha_{k+1} \leq k+2$, then $\alpha_k \leq 2 + k^2/(k+2)$, which is at most $k+1$ if and only if $2k+4 \leq 3k+2$, that is, $k \geq 2$. □

By (7) and (6),

$$\Phi_{k+1} \leq E_k + c_k\,\delta_k + \alpha_{k+1}\big(h_k + (1-h_k)W_k\big), \qquad c_k := \frac{2(k+1)}{k+2} - \alpha_{k+1}h_k(1-h_k).$$

Since $h_k(1-h_k) = 2k/(k+2)^2$ and $\alpha_{k+1} \leq k+2$ (Lemma 3, or $\alpha_N = 0$), we have $c_k \geq 2/(k+2) > 0$, so $\delta_k \leq W_k$ yields

$$\Phi_{k+1} \leq E_k + \alpha_{k+1}h_k + W_k\big(c_k + \alpha_{k+1}(1-h_k)\big) = \Phi_k + \alpha_{k+1}h_k,$$

because $c_k + \alpha_{k+1}(1-h_k) = \frac{2(k+1)}{k+2} + \alpha_{k+1}(1-h_k)^2 = \alpha_k$. Summing over $k = 1, \ldots, N-1$, and using $\Phi_N = E_N$ and $\Phi_1 = E_1 + \alpha_1 W_1 \leq 1 + \alpha_1$ (Step 3),

$$E_N \leq B_N := 1 + \alpha_1 + \sum_{k=1}^{N-1} \alpha_{k+1}h_k. \tag{9}$$

**Step 5: evaluating $B_N$.** Rather than solve (8) directly, we evaluate $B_N$ on the path that makes every inequality tight. Define numbers $\bar{E}_1 = \bar{W}_1 = 1$ and $\bar{W}_{k+1} = h_k + (1-h_k)^2\bar{W}_k$, $\bar{E}_{k+1} = \bar{E}_k + \frac{2(k+1)}{k+2}\bar{W}_k$. These satisfy all relations of Step 4 with equality (take $\delta_k = \bar{W}_k$), hence $\bar{E}_N = B_N$. By induction,

$$\bar{W}_k = \frac{(k+2)(3k+1)}{6k(k+1)},$$

since $\frac{2}{k+2} + \frac{k^2}{(k+2)^2} \cdot \frac{(k+2)(3k+1)}{6k(k+1)} = \frac{3k^2+13k+12}{6(k+1)(k+2)} = \frac{(k+3)(3k+4)}{6(k+1)(k+2)}$. Therefore $\frac{2(k+1)}{k+2}\bar{W}_k = \frac{3k+1}{3k} = 1 + \frac{1}{3k}$ and

$$B_N = \bar{E}_N = 1 + \sum_{k=1}^{N-1}\Big(1 + \frac{1}{3k}\Big) = N + \tfrac{1}{3}H_{N-1}.$$

Combining with (9), $e_N = E_N/(N(N+1)) \leq (N + \frac{1}{3}H_{N-1})/(N(N+1))$. □

*Remark* 4 (What the argument captures, and what it misses). Inequalities (3)–(5) define a linear program in $(\delta_k)$ whose value is exactly $B_N/(N(N+1))$. Its worst case saturates $\delta_k = 1 - V_k$, so the spread of the cloud tends to $D^2/2$. In the exact PEP of Frank–Wolfe, the improvement over $2LD^2/N$ comes almost entirely from the pairwise diameter constraints among $\{x_0, s_0, s_1, \dots\}$. Dropping them restores a leading constant close to 2, while dropping the cross linear-minimization constraints or the optimality constraints of $x^\star$ hardly changes the value. The variance identity is a convenient aggregate of exactly those pairwise constraints, which is why a single number can capture most of the improvement. For the step sizes $h_k = \ell/(k+\ell)$ the same argument gives the smallest asymptotic constant at $\ell = 2$.

## 3 Consequence for the Frank–Wolfe gap

The error $f(x_k) - f^\star$ cannot be computed without knowing $f^\star$. The Frank–Wolfe (duality) gap

$$g_k = \max_{s \in C} \langle \nabla f(x_k), x_k - s \rangle = \langle \nabla f(x_k), x_k - s_k \rangle$$

can: the method computes $s_k$ anyway. By convexity $g_k \geq f(x_k) - f^\star$, so the gap is the standard stopping certificate, and solvers stop once it falls below a tolerance. A guarantee on the gap therefore bounds how long a solver runs. The classical guarantee is $\min_{k \leq N} g_k \leq \frac{27}{4} LD^2/(N+2)$ [2, Thm. 2].

**Corollary 5.** *For every $N \geq 1$, $\min_{0 \leq k \leq N} g_k \leq \dfrac{3\,LD^2}{N+1}$.*

*Proof.* Take $L = D = 1$. Smoothness gives $h_k g_k \leq e_k - e_{k+1} + h_k^2/2$. Summing over $K_0 \leq k \leq N$ and using $e_{N+1} \geq 0$,

$$\Big(\min_{K_0 \leq k \leq N} g_k\Big) \sum_{k=K_0}^{N} h_k \;\leq\; e_{K_0} + \tfrac{1}{2} \sum_{k=K_0}^{N} h_k^2,$$

and we bound $e_{K_0}$ by Theorem 1.

For $N \leq 400$, optimizing over $K_0$ in exact rational arithmetic gives $(N+1)\min_k g_k \leq 2.8984 < 3$. For $N > 400$, take $m = K_0 = \lceil N/2 \rceil \geq 200$. Then

- $\frac{1}{2}\sum_{k \geq m} h_k^2 \leq \frac{2}{m+1} - \frac{2}{N+2}$;
- $e_m \leq \frac{1}{m+1}\big(1 + \frac{1+\ln m}{3m}\big) \leq \frac{1.0105}{m+1}$;
- $\frac{1}{m+1} \leq \frac{2}{N+2}$.

So the right-hand side is at most $4.021/(N+2)$. Moreover $\sum_{k=m}^{N} h_k \geq 2 \ln \frac{N+3}{m+2} \geq 2 \ln \frac{2 \cdot 403}{405} \geq 1.3763$. Hence $\min_k g_k \leq 2.9217/(N+2) < 3/(N+1)$. □

The asymptotic constant of this argument is $\min_{\theta \in (0,1)} \frac{3/\theta - 2}{2\ln(1/\theta)} \approx 2.882$. If the variance constraint is also used inside the sum (a linear program over $e_k, g_k, \delta_k, V_k$), numerically $(N+2)\min_k g_k \to \approx 2.15$, against $\approx 4.30$ for the classical relaxation $\delta_k \leq D^2$. The PEP value is $\approx 1.67$.

## 4 How close is the bound to the truth?

We computed the exact worst-case value $w_N = \sup\,(f(x_N) - f^\star)/(LD^2)$ by solving the tight PEP [8]. We used the interpolation conditions for $L$-smooth convex functions together with $C = \mathrm{conv}\{x_0, s_0, \ldots, s_{N-1}, x^\star\}$ (linear-oracle optimality of each $s_k$, optimality of $x^\star$ with respect to all vertices, and all pairwise vertex distances at most $D$), and solved with the interior-point solver Clarabel. Table 1 lists the values and Figure 2 plots the constants. These values are exact for functions that are $L$-smooth on all of $\mathbb{R}^d$. Theorem 1 needs smoothness only on $C$; for that larger class the values are lower bounds on the worst case.

| $N$ | PEP $w_N$ | $Nw_N$ | Thm. 1 | Jaggi $2/(N+2)$ |
|---|---|---|---|---|
| 1 | 0.50000 | 0.500 | 0.50000 | 0.66667 |
| 2 | 0.31262 | 0.625 | 0.38889 | 0.50000 |
| 3 | 0.22785 | 0.684 | 0.29167 | 0.40000 |
| 5 | 0.14621 | 0.731 | 0.18981 | 0.28571 |
| 10 | 0.07829 | 0.783 | 0.09948 | 0.16667 |
| 20 | 0.04132 | 0.826 | 0.05043 | 0.09091 |
| 40 | 0.02156 | 0.863 | 0.02525 | 0.04762 |
| 60 | 0.01468 | 0.881 | 0.01682 | 0.03226 |
| 80 | 0.01116 | 0.893 | 0.01260 | 0.02439 |

Table 1: Exact worst case (PEP, numerical), the bound of Theorem 1, and the classical bound ($L = D = 1$). At $N = 3$ the PEP value agrees with $18/79$ to eight digits. For $N = 60$ and $N = 80$ the solver reported reduced accuracy.

The sequence $Nw_N$ increases slowly, and extrapolation suggests a limit near 0.9–0.95. The exact constant $c^\star = \lim_N Nw_N$ lies in $[1/4, 1]$ by [7] and Theorem 1. Determining it remains open. Remark 4 suggests where to look. The variance identity sums the pairwise distance constraints into one number; the gap between our constant 1 and the PEP value of about 0.9 presumably comes from using them one by one.

## Reproducibility

All computations are in the accompanying repository, `https://github.com/samsonmaximus/Frank-Wolfe-Constant-Halved`, under `experiments/` (Python 3.11, numpy, scipy, cvxpy with Clarabel):

- `pep_fw.py`: the exact PEP, which produces the values $w_N$ of Table 1;
- `pep_fw_ablate.py`, `pep_fw_local.py`: the constraint-ablation experiments described in Remark 4;
- `variance_lp.py`: the linear program defined by (3)–(5), for general $\ell$;
- `check_proof.py`: exact rational checks of (8), of Lemma 3, of $c_k > 0$ and of $B_N = N + \frac{1}{3}H_{N-1}$ (for $N \le 199$ and $N = 500, 1000$), and random feasible paths;
- `referee_fw.py`: an independent check, which runs Frank–Wolfe on random and adversarial instances;

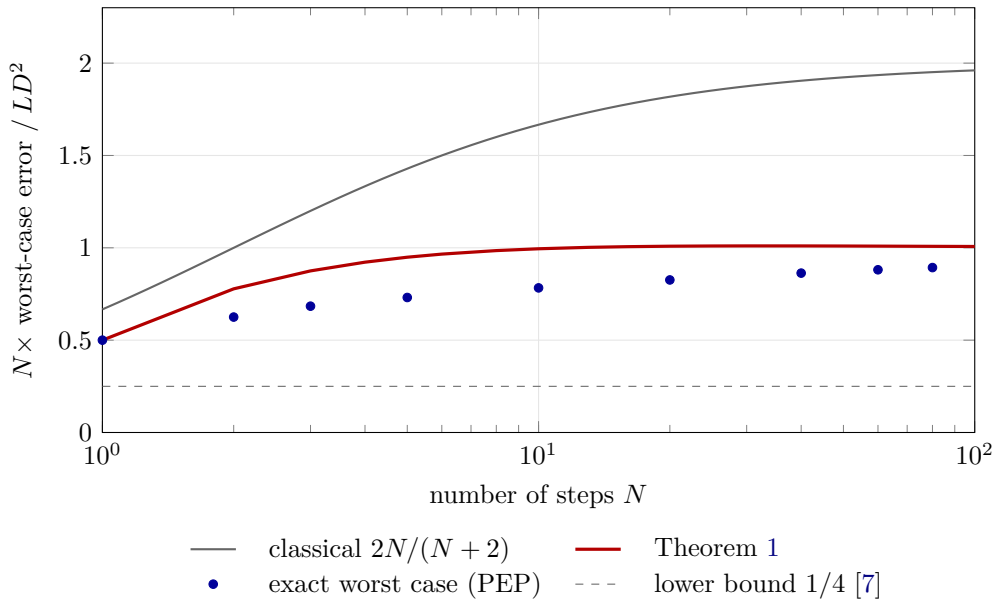



Figure 2: The constant in the Frank–Wolfe rate: $N$ times the worst-case error after $N$ steps, in units of $LD^2$. The classical bound tends to 2, Theorem 1 to 1, and the exact worst case appears to approach about 0.9. Lan's lower bound holds for every linear-minimization method.

- `gap_corollary_all.py`: the exact rational check of Corollary 5 for every $N \le 400$; `gap_lp.py`: the variance relaxation for the gap.

## Acknowledgements

The proof was found and checked with the help of AI-assisted computational tools. All arguments have been verified line by line, and the numerical claims can be reproduced with the scripts above.

## References

[1] M. Frank and P. Wolfe. An algorithm for quadratic programming. *Naval Res. Logist. Quart.* 3 (1956) 95–110.

[2] M. Jaggi. Revisiting Frank–Wolfe: projection-free sparse convex optimization. ICML 2013.

[3] R. M. Freund and P. Grigas. New analysis and results for the Frank–Wolfe method. *Math. Program.* 155 (2016). arXiv:1307.0873.

[4] G. Braun, A. Carderera, C. W. Combettes, H. Hassani, A. Karbasi, A. Mokhtari, S. Pokutta. Conditional gradient methods. arXiv:2211.14103.

[5] F. Bach. Duality between subgradient and conditional gradient methods. *SIAM J. Optim.* 25 (2015). arXiv:1211.6302.

[6] Yu. Nesterov. Complexity bounds for primal-dual methods minimizing the model of objective function. *Math. Program.* 171 (2018).

[7] G. Lan. The complexity of large-scale convex programming under a linear optimization oracle. arXiv:1309.5550.

[8] A. B. Taylor, J. M. Hendrickx, F. Glineur. Exact worst-case performance of first-order methods for composite convex optimization. *SIAM J. Optim.* 27 (2017). arXiv:1512.07516.

[9] A. Luner and B. Grimmer. Performance estimation for smooth and strongly convex sets. arXiv:2410.14811.